

4/22/2026

Virtual Stock Market

Submitted By:

Name: Krushal Patel

USN No: 2410111714

Name: Vir Khemka

USN No: 2410111910

Division: CSE-B Batch: B1

Submitted To:

Ms. Nidhi Acharya

Academic Year:2026-2027

CASE STUDY: Virtual Stock Market

Abstract:

The Virtual Stock Market is a web-based stock trading simulation platform developed to provide users with practical knowledge of stock market operations without financial risk. The project is designed using React.js for the frontend user interface and Java Spring Boot for the backend services. It simulates real-time stock market behavior by displaying dynamic stock prices, percentage changes, trading volume, and market capitalization.

Each user receives an initial virtual wallet balance of ₹20,000, which can be used to buy and sell shares of various listed companies such as Reliance, TCS, Infosys, Airtel, HDFC Bank, and others. The system automatically updates wallet balance, personal portfolio, and transaction history after every trade. Users can also view individual stock pages with graphical price trends for different durations such as 1 Day, 1 Month, 1 Year, 2 Years, 5 Years, and All Time.

The platform includes additional features such as live market news, profit/loss tracking, stock performance indicators, and separate personal portfolios for multiple users through login authentication. Positive and negative market changes are visually represented using color indicators to enhance user understanding.

The main objective of this project is to help beginners, students, and aspiring investors learn stock trading concepts, portfolio management, and market analysis in a realistic virtual environment. It also demonstrates practical implementation of full-stack development concepts such as REST APIs, object-oriented programming, database integration, and dynamic user interfaces.

Functional Requirements

1.Create an account and log in:

The system will enable new users such as creating an account with a username, email and a password.

The system will enable the current users to be able to log in safely.

The system will keep distinct data of every user.

2. User Wallet Management:

The system will allow each new user to have an initial virtual balance of 20,000.

The system will show the balance of the wallet at the moment.

During the purchase of stocks, the system will automatically deduct the money.

When stocks are sold, the system will automatically add money to the system.

3. Stock Dashboard:

A list of stocks available shall appear in the system.

The system will indicate the name of stock, price, change percentage, volume and market capitalization.

The system will dynamically update stock prices. The system will indicate gainers with green and losses with red.

4. Buy Stock Functionality:

The system will enable users to choose a stock.

The system will enable users to key in quantity.

This system will compute total price prior to purchase.

The system will ensure that there is adequate wallet balance.

The system will include bought stock to the portfolio.

5. Sell Stock Functionality:

The system will enable the users to sell stocks that they hold.

The system will check the quantity available and then sell.

The system will increase wallet balance with sale amount.

The system will refresh the portfolio following selling.

6. Portfolio Management:

The system will show stocks purchased.

The system will display: stock name, quantity, buy price, current price, current value, and profit/loss.

The system will be updated with profit/loss on live market price.

7. Transaction History:

The system will have full trade history.

The system will document buy/sell, stock name, quantity, price and date/time.

The system will show the recent transactions in the descending order.

8. Stock Detail Page:

The system will open a dissimilar page on chosen stock.

The system will indicate stock graph with time filters: 1 Day 1 Month 1 Year 2 Years 5 Years All Time

9. Live Market News:

System will present simulated live market news.

News headlines will be updated periodically in the system. The system will be visual with positive or negative impact.

10. Multi-User Data Separation:

The system is to have personal wallets to each user.

The system will uphold different portfolio and histories of individual users.

The information of one user will not be accessible to another user.

11. Backend API Services:

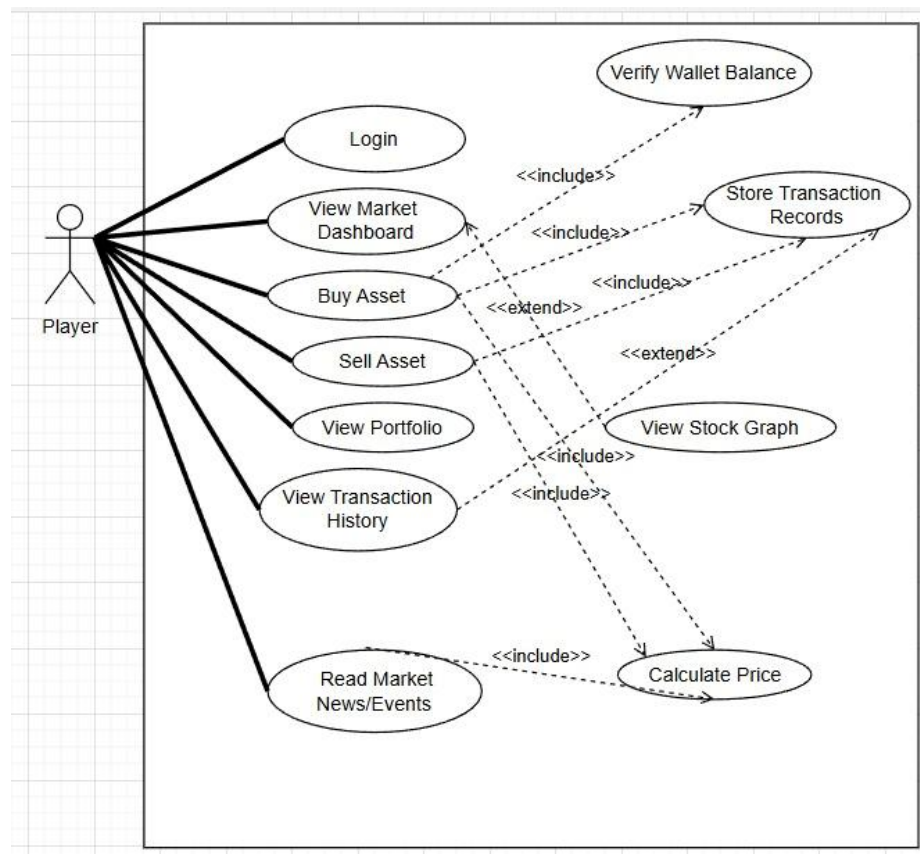
The system will have REST APIs on the frontend.

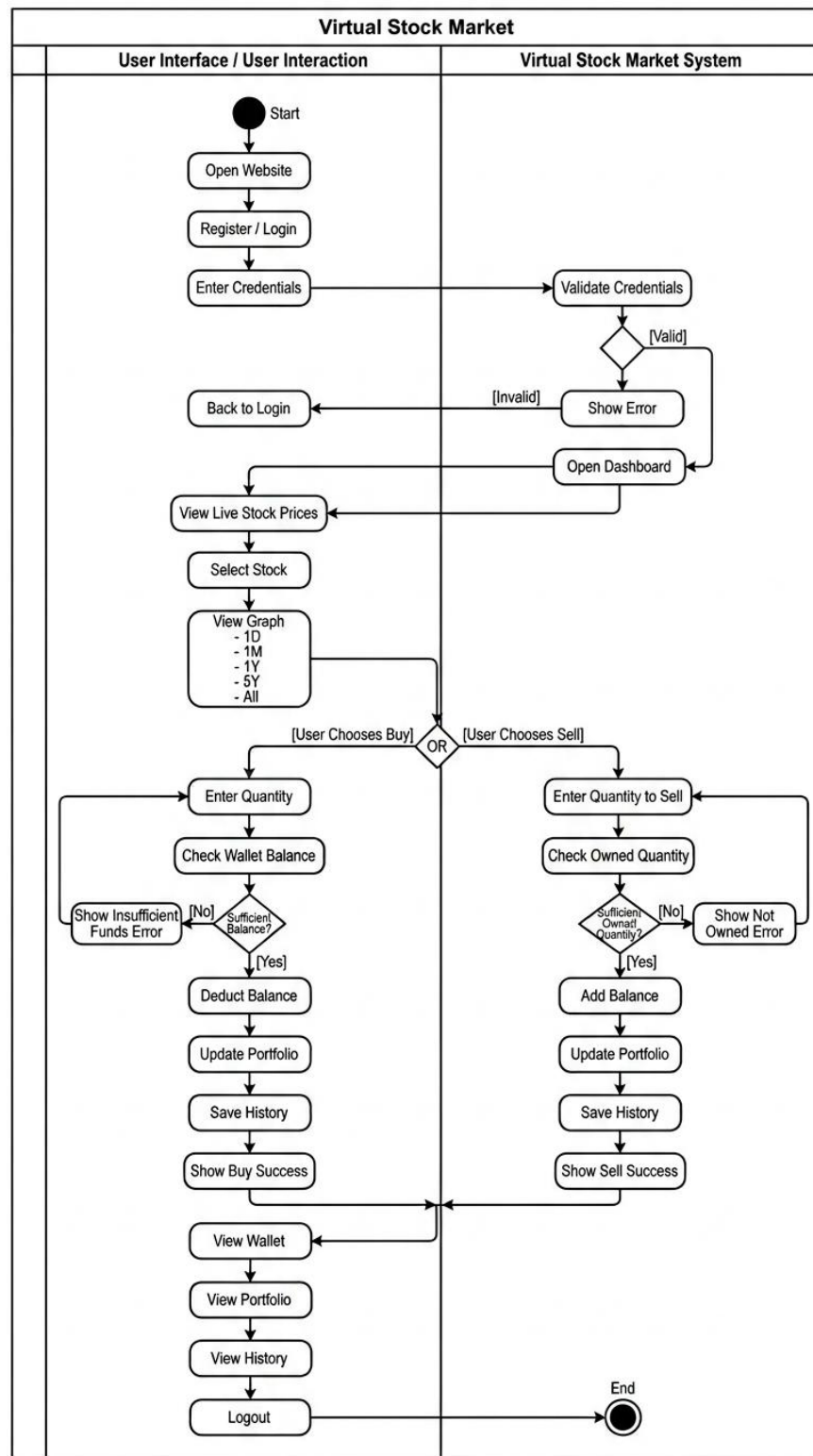
The system will be enabled with stock, wallet, portfolio, history, buy and sell APIs.

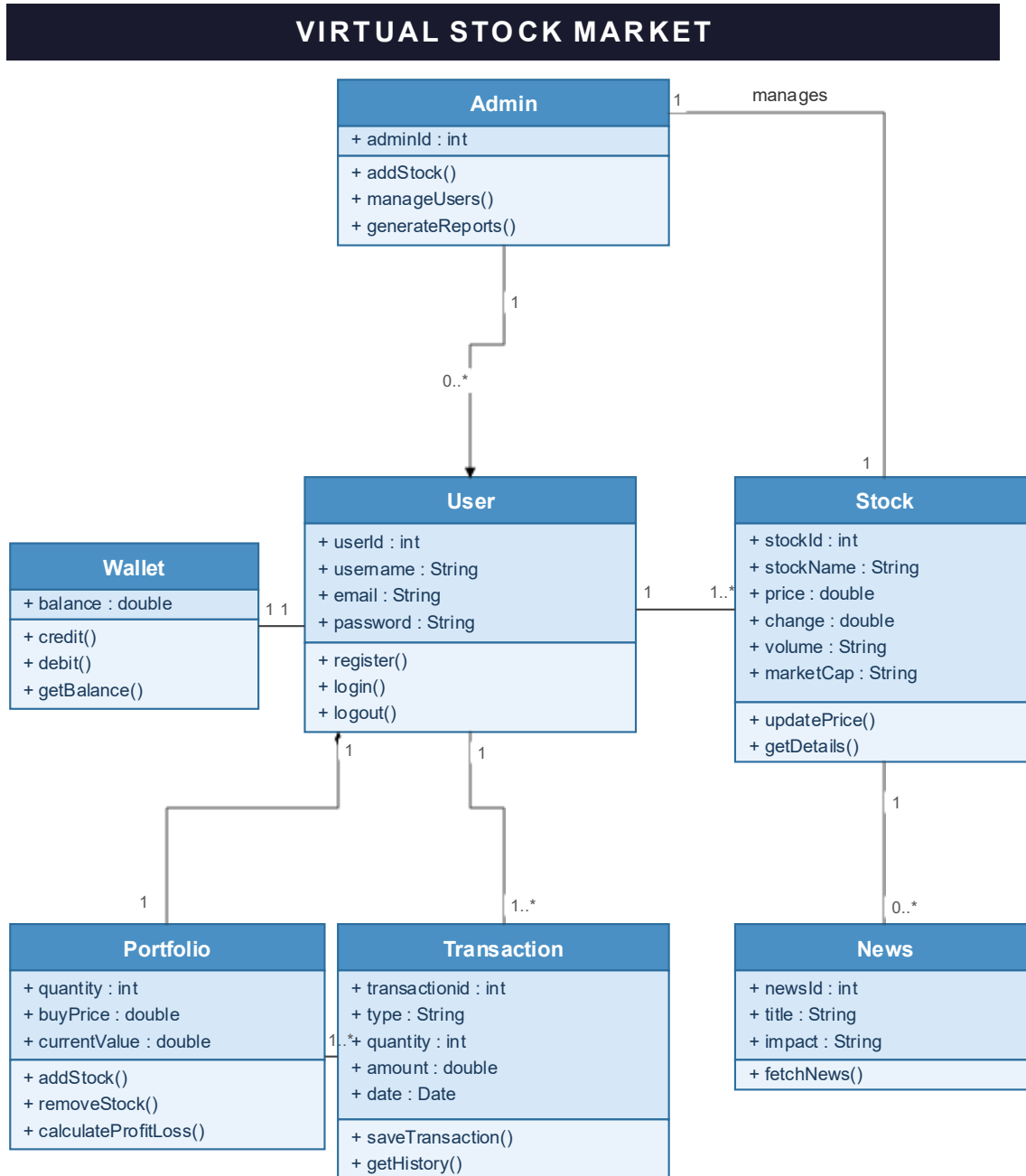
12. Error Handling: The system will show an appropriate error message of: Insufficient wallet balance
Invalid quantity Selling unavailable stock Login failure Server connection issue

UML Diagram:

Use Case:



Activity Diagram :

Class Diagram :

Structure or Flow Diagram :**COMPLETE OPTIMAL FLOW — REQUEST TO RESPONSE**

User → Browser/App → React Frontend → REST API Call (HTTP GET/POST, JSON) → Spring Boot Controller → Service Layer → Business Logic → ArrayList/Variables → Response (JSON) → React UI Update → User sees result

BUY STOCK — OPTIMAL PATH

1. User opens Dashboard → sees live stock list
2. Clicks on a stock → Stock Detail Page opens
3. Enters quantity → Total Value calculated automatically
4. Clicks Buy → Frontend sends POST request to /api/stocks/buy
5. WalletController receives request → calls WalletService
6. WalletService checks wallet balance against total cost
7. If balance sufficient → deducts amount from Wallet Balance variable
8. PortfolioService adds stock entry to List<Portfolio>
9. HistoryService saves transaction to List<Transaction>
10. Response JSON sent back → React updates Wallet, Portfolio, and History pages
11. UI shows success confirmation

SELL STOCK — OPTIMAL PATH

1. User opens Portfolio Page → sees owned stocks
2. Selects stock to sell → Stock Detail Page opens
3. Enters quantity to sell
4. Clicks Sell → Frontend sends POST to /api/stocks/sell
5. PortfolioController receives → calls PortfolioService
6. PortfolioService checks List<Portfolio> for sufficient quantity
7. If quantity available → removes/reduces entry in portfolio
8. WalletService credits sale amount to Wallet Balance variable
9. HistoryService saves sell transaction to List<Transaction>
10. Response JSON sent back → React updates Portfolio, Wallet, History
11. UI shows sell success

LOGIN — OPTIMAL PATH

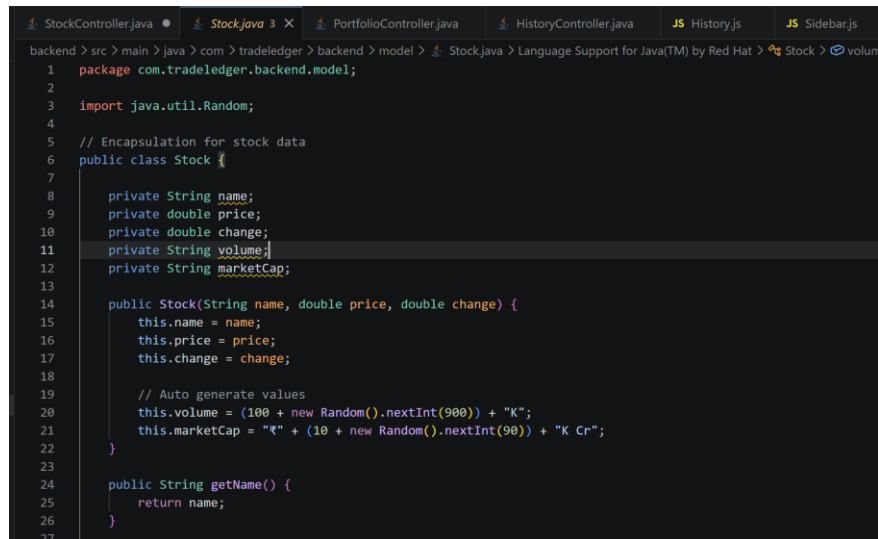
1. User enters email + password → clicks Login
2. React sends POST to /api/users/login
3. UserController receives → UserService validates credentials
4. Matches against User Session Variable in memory
5. If valid → session created, Dashboard loads
6. If invalid → error response → React shows error message on Login Page

Git Hub Link: <https://github.com/KrushaPatel1310/TradeLedger>

Input: all the OOP Concepts in code

OOP Concept	Used In Files
Class	BackendApplication.java, Stock.java, User.java, Wallet.java, Transaction.java, StockController.java, WalletController.java, PortfolioController.java, HistoryController.java, StockService.java, WalletService.java, PortfolioService.java, HistoryService.java
Object	StockService.java, PortfolioService.java, BackendApplication.java
Encapsulation	Stock.java, User.java, Wallet.java, Transaction.java, WalletService.java, PortfolioItem.java
Constructor	Stock.java, User.java, Wallet.java, Transaction.java, PortfolioItem.java
Abstraction	StockService.java, WalletService.java, PortfolioService.java, HistoryService.java, StockController.java, WalletController.java, PortfolioController.java, HistoryController.java
Inheritance	Spring Boot internal classes, BackendApplication.java (framework inheritance usage)
Polymorphism	StockService.java, PortfolioService.java, Controller classes, List = ArrayList
Interface	StockService.java, PortfolioService.java, HistoryService.java (List)
Method Overloading	Stock.java (if multiple constructors), Service files (if multiple buy/sell methods)
Method Overriding	Spring framework internal controllers/services
Association	StockController.java ↔ StockService.java, WalletController.java ↔ WalletService.java, PortfolioController.java ↔ PortfolioService.java, HistoryController.java ↔ HistoryService.java
Aggregation	StockService.java (List<Stock>), PortfolioService.java (List<PortfolioItem>), HistoryService.java (List<String>)
Composition	User.java + Wallet.java, User.java + Portfolio
Access Modifiers	Almost all Java files (private, public)
Package Concept	All files

1. Defining a class and creating objects:

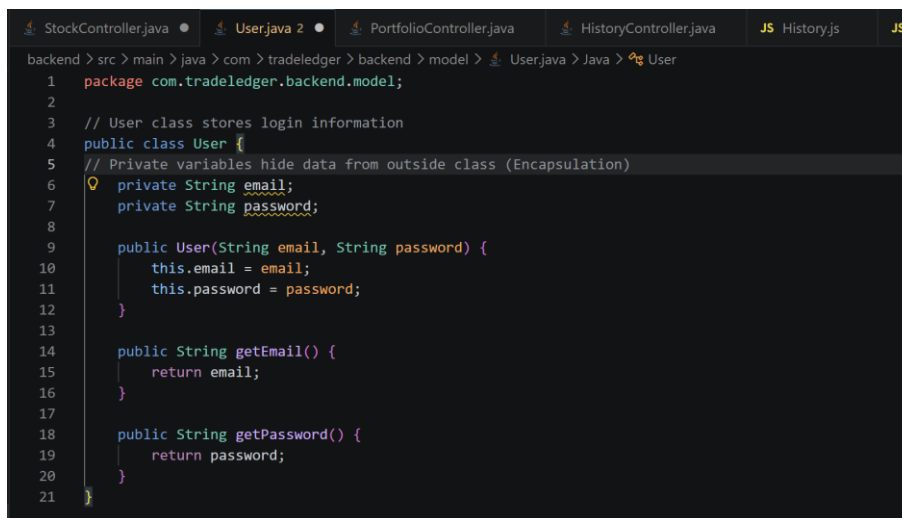


```

1 package com.tradedledger.backend.model;
2
3 import java.util.Random;
4
5 // Encapsulation for stock data
6 public class Stock {
7
8     private String name;
9     private double price;
10    private double change;
11    private String volume;
12    private String marketCap;
13
14    public Stock(String name, double price, double change) {
15        this.name = name;
16        this.price = price;
17        this.change = change;
18
19        // Auto generate values
20        this.volume = (100 + new Random().nextInt(900)) + "K";
21        this.marketCap = "$" + (10 + new Random().nextInt(90)) + "K Cr";
22    }
23
24    public String getName() {
25        return name;
26    }
27

```

2. Encapsulation Private fields + public getters/setters

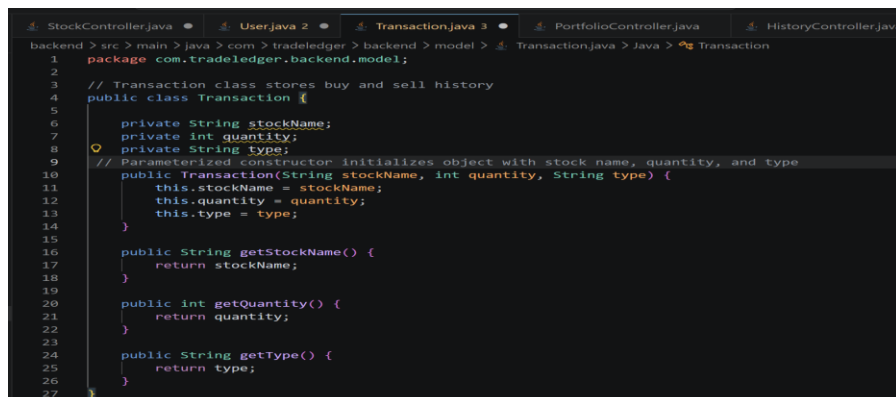


```

1 package com.tradedledger.backend.model;
2
3 // User class stores login information
4 public class User {
5     // Private variables hide data from outside class (Encapsulation)
6     private String email;
7     private String password;
8
9     public User(String email, String password) {
10        this.email = email;
11        this.password = password;
12    }
13
14    public String getEmail() {
15        return email;
16    }
17
18    public String getPassword() {
19        return password;
20    }
21

```

3. Constructor Parameterized constructors initialising objects

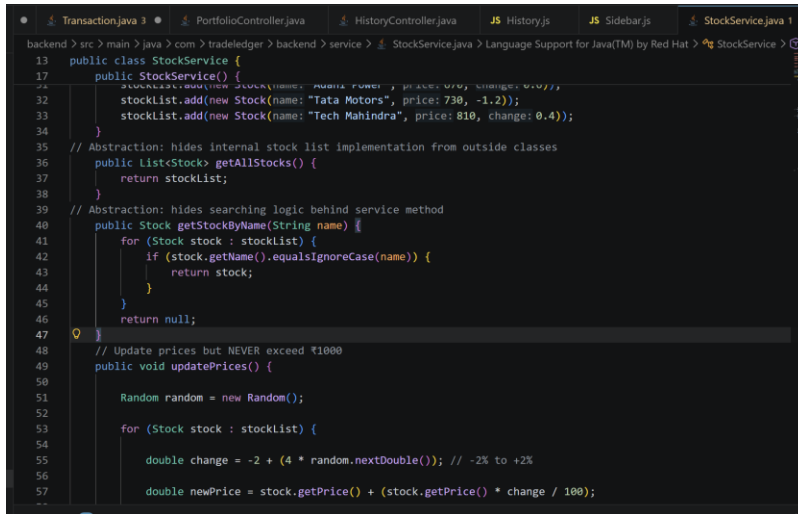


```

1 package com.tradedledger.backend.model;
2
3 // Transaction class stores buy and sell history
4 public class Transaction {
5
6     private String stockName;
7     private int quantity;
8     private String type;
9     // Parameterized constructor initializes object with stock name, quantity, and type
10    public Transaction(String stockName, int quantity, String type) {
11        this.stockName = stockName;
12        this.quantity = quantity;
13        this.type = type;
14    }
15
16    public String getStockName() {
17        return stockName;
18    }
19
20    public int getQuantity() {
21        return quantity;
22    }
23
24    public String getType() {
25        return type;
26    }
27

```

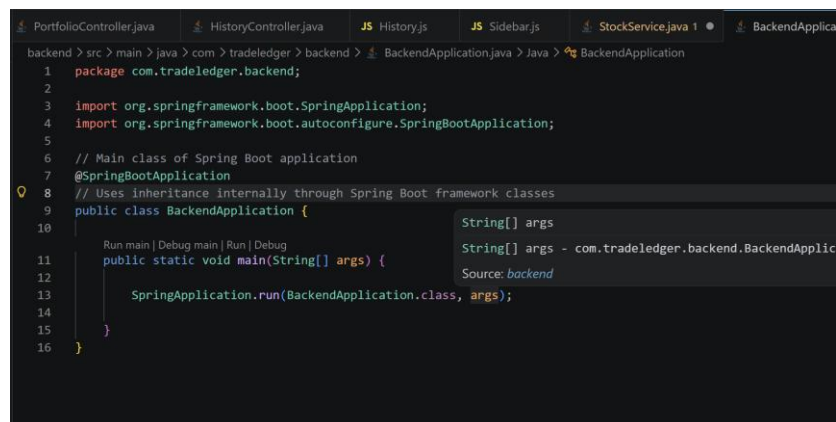
4. Abstraction Hiding implementation behind service methods



```

13 public class StockService {
14     // Abstraction: hides internal stock list implementation from outside classes
15     public List<Stock> getAllStocks() {
16         return stockList;
17     }
18
19     // Abstraction: hides searching logic behind service method
20     public Stock getStockByName(String name) {
21         for (Stock stock : stockList) {
22             if (stock.getName().equalsIgnoreCase(name)) {
23                 return stock;
24             }
25         }
26         return null;
27     }
28
29     // Update prices but NEVER exceed ₹1000
30     public void updatePrices() {
31         Random random = new Random();
32
33         for (Stock stock : stockList) {
34             double change = -2 + (4 * random.nextDouble()); // -2% to +2%
35             double newPrice = stock.getPrice() + (stock.getPrice() * change / 100);
36         }
37     }
38 }
  
```

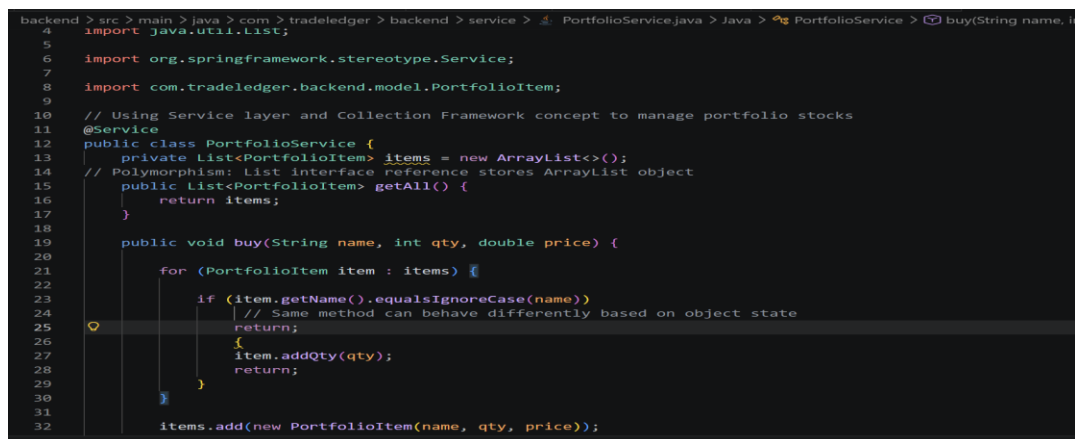
5. Inheritance Spring Boot framework inheritance



```

1 package com.tradedledger.backend;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 // Main class of Spring Boot application
7 @SpringBootApplication
8 // Uses inheritance internally through Spring Boot framework classes
9 public class BackendApplication {
10     public static void main(String[] args) {
11         SpringApplication.run(BackendApplication.class, args);
12     }
13 }
  
```

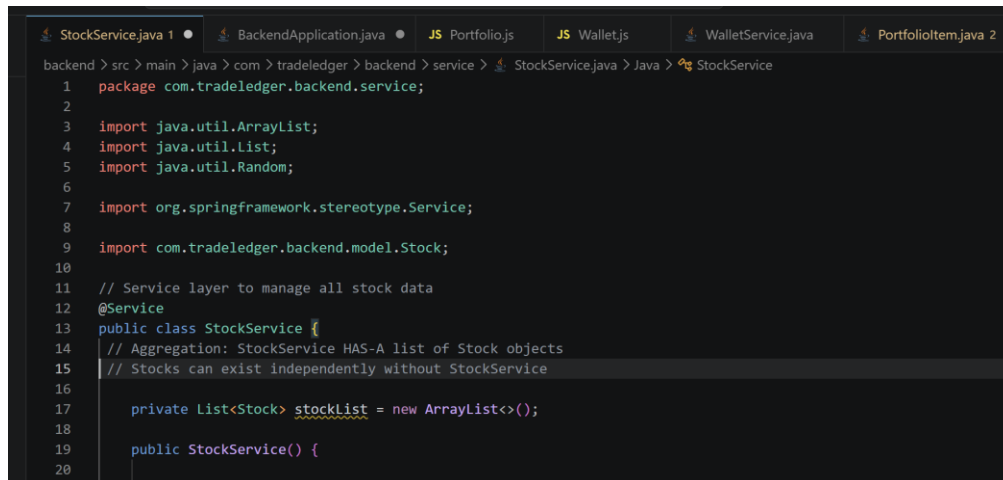
6. Polymorphism Same method name, different behaviour



```

4 import java.util.List;
5
6 import org.springframework.stereotype.Service;
7
8 import com.tradedledger.backend.model.PortfolioItem;
9
10 // Using Service layer and Collection Framework concept to manage portfolio stocks
11 @Service
12 public class PortfolioService {
13     private List<PortfolioItem> items = new ArrayList<>();
14     // Polymorphism: List interface reference stores ArrayList object
15     public List<PortfolioItem> getAll() {
16         return items;
17     }
18
19     public void buy(String name, int qty, double price) {
20         for (PortfolioItem item : items) {
21             if (item.getName().equalsIgnoreCase(name)) {
22                 // Same method can behave differently based on object state
23                 return;
24             }
25             item.addQty(qty);
26             return;
27         }
28         items.add(new PortfolioItem(name, qty, price));
29     }
30 }
  
```

7. Aggregation Service holds a collection of independent objects

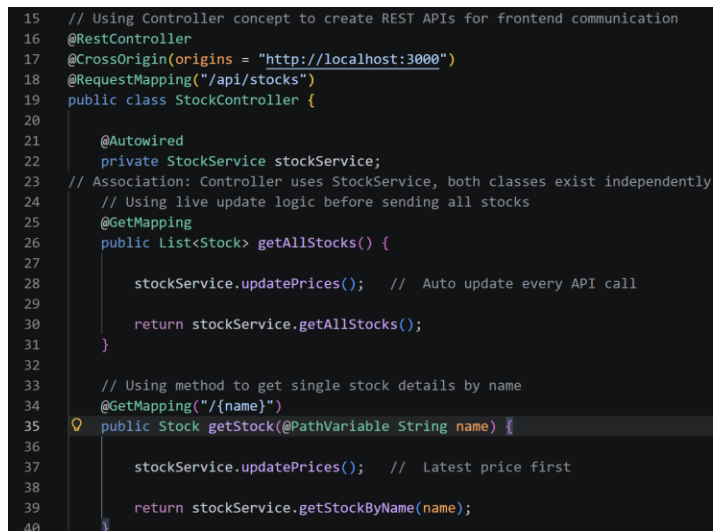


```

1  package com.tradedledger.backend.service;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Random;
6
7  import org.springframework.stereotype.Service;
8
9  import com.tradedledger.backend.model.Stock;
10
11 // Service layer to manage all stock data
12 @Service
13 public class StockService {
14     // Aggregation: StockService HAS-A list of Stock objects
15     // Stocks can exist independently without StockService
16
17     private List<Stock> stockList = new ArrayList<>();
18
19     public StockService() {
20

```

8. Association Controller uses Service — both exist independently

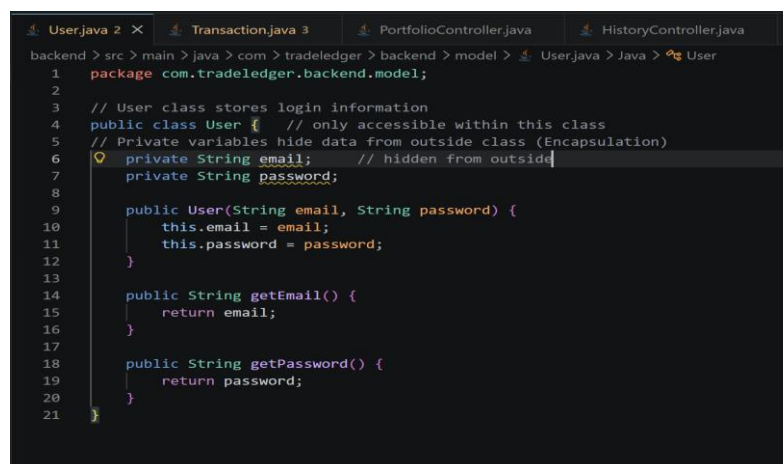


```

15 // Using Controller concept to create REST APIs for frontend communication
16 @RestController
17 @CrossOrigin(origins = "http://localhost:3000")
18 @RequestMapping("/api/stocks")
19 public class StockController {
20
21     @Autowired
22     private StockService stockService;
23     // Association: Controller uses StockService, both classes exist independently
24     // Using live update logic before sending all stocks
25     @GetMapping
26     public List<Stock> getAllStocks() {
27
28         stockService.updatePrices(); // Auto update every API call
29
30         return stockService.getAllStocks();
31     }
32
33     // Using method to get single stock details by name
34     @GetMapping("/{name}")
35     public Stock getStock(@PathVariable String name) {
36
37         stockService.updatePrices(); // Latest price first
38
39         return stockService.getStockByName(name);
40     }

```

9. Access Modifiers private, public, protected in action



```

1  package com.tradedledger.backend.model;
2
3  // User class stores login information
4  public class User { // only accessible within this class
5      // Private variables hide data from outside class (Encapsulation)
6      private String email; // hidden from outside
7      private String password;
8
9      public User(String email, String password) {
10         this.email = email;
11         this.password = password;
12     }
13
14     public String getEmail() {
15         return email;
16     }
17
18     public String getPassword() {
19         return password;
20     }
21 }

```

10. Package Concept Organising classes into packages

```

1  package com.tradeledger.backend.controller;
2
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.web.bind.annotation.CrossOrigin;
5  import org.springframework.web.bind.annotation.GetMapping;
6  import org.springframework.web.bind.annotation.RequestMapping;
7  import org.springframework.web.bind.annotation.RequestParam;
8  import org.springframework.web.bind.annotation.RestController;
9
10 import com.tradeledger.backend.service.HistoryService;
11 import com.tradeledger.backend.service.PortfolioService;
12 import com.tradeledger.backend.service.WalletService;
13
14 // Using Controller and REST API concept to handle buy and sell stock requests
15 @RestController
16 @RequestMapping("/api/trade")
17 @CrossOrigin(origins = "http://localhost:3000")
18 public class TradeController {
19

```

Output: Put all the SS with fig number and name

Figure 1: User Login Page:

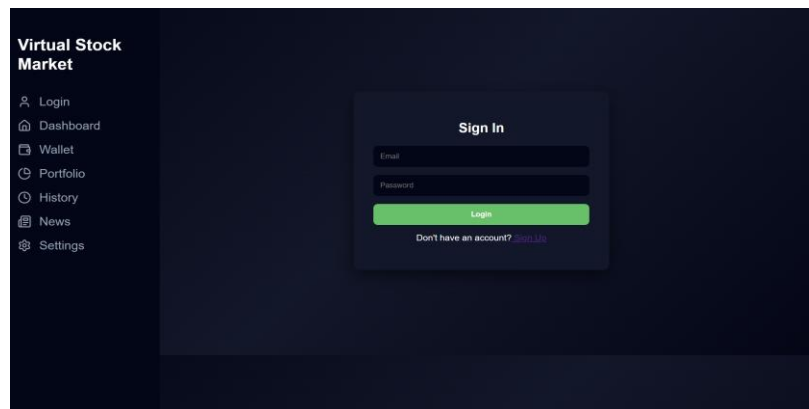


Figure 2: Dashboard Page

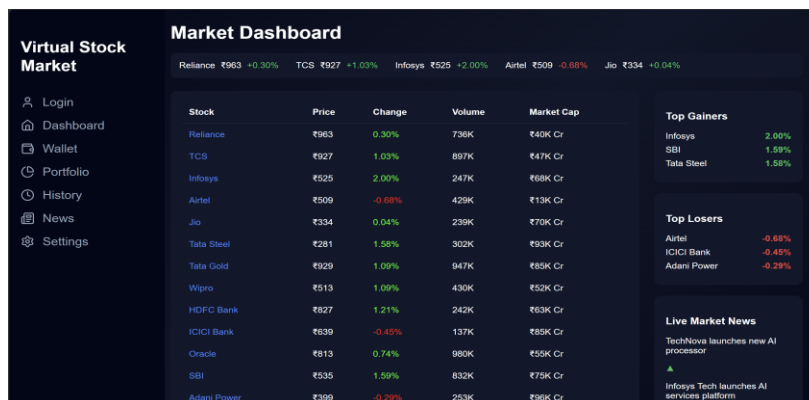
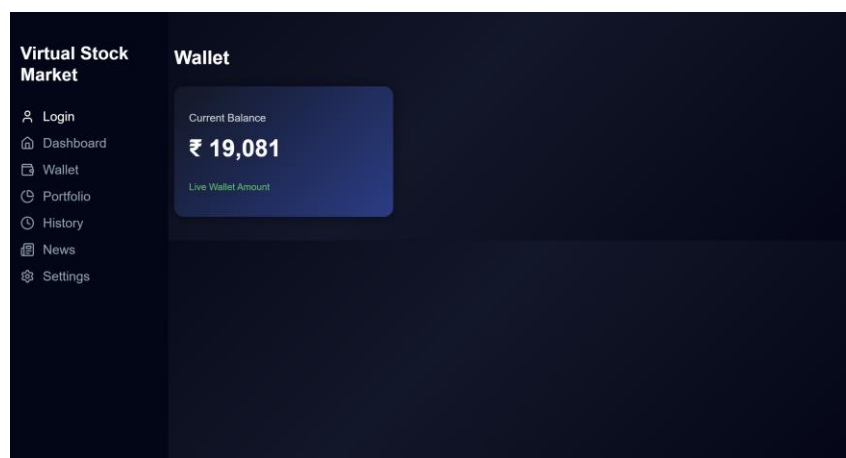
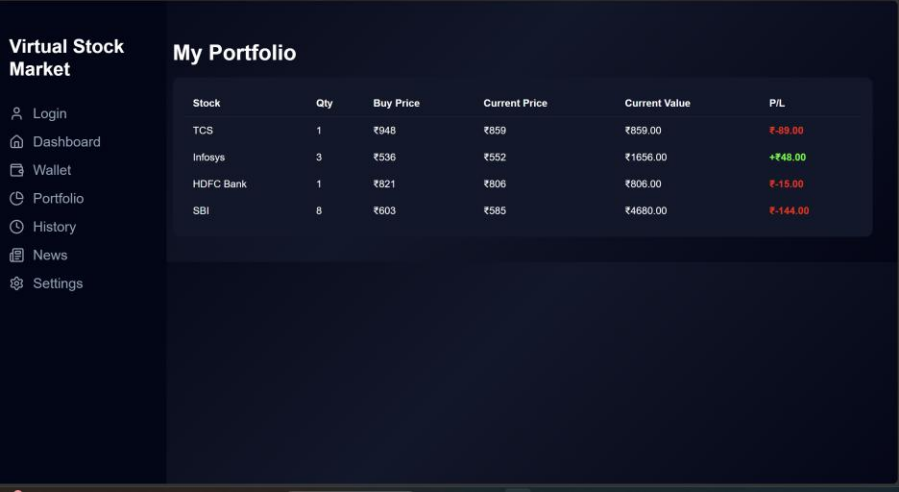


Figure 3: Stock Detail Page (same for the all stock)**Figure 3.1: Stock Detail Page (graph day ,month ,year ,all)****Figure 4 : Wallet Page****Figure 5 : Portfolio Page**



Stock	Qty	Buy Price	Current Price	Current Value	P/L
TCS	1	₹948	₹859	₹859.00	₹-89.00
Infosys	3	₹536	₹552	₹1656.00	+₹48.00
HDFC Bank	1	₹821	₹806	₹806.00	₹-15.00
SBI	8	₹603	₹585	₹4680.00	₹-144.00

Figure 6 : Transaction History Page



Type	Stock	Quantity	Status
BUY	SBI	8	Bought
BUY	HDFC	Qty	Bought
BUY	Infosys	3	Bought
SELL	TCS	19	Sold
SELL	Tata	Qty	Sold
BUY	Tata	Qty	Bought
BUY	TCS	20	Bought

Conclusion:

The Virtual Stock Market project was a success that created an interactive and realistic simulation of a stock trading platform with the help of React.js as a front-end and Java Spring Boot as a back-end. It enables users to simulate stock market activities (live stock prices, buy/sell stocks, manage wallet, track portfolio, check the transaction history and examine stock trend in graphic representations) without financial risk. The project realizes significant concepts like Object-Oriented Programming, REST APIs, modular design, and dynamic design of the UI. It enables users to learn the real-world investment strategies, computing profits and losses and market characteristics within a virtual setting. On the whole, this project is an educational, scalable solution, which is feasible and can be expanded with database connectivity, real-time APIs, and AI-based market analysis in the future.

Future Enhancement:

1. MySQL Database Integration for permanent storage of users, portfolio, wallet, and transaction history.
2. Real-Time Stock API Integration to fetch live market prices from NSE/BSE or global exchanges.

3. AI-Based Stock Prediction System for suggesting future stock trends and investment opportunities.
4. Mobile Application Development for Android and iOS platforms.
5. Advanced Graph Analytics with candlestick charts, indicators, and technical analysis tools.
6. Secure Authentication System using JWT, OTP, and email verification.
7. Multi-User Leaderboard to compare portfolio performance among users.
8. Admin Panel Dashboard to manage users, stocks, reports, and system settings.
9. News Sentiment Analysis to automatically impact stock prices based on positive or negative news.
10. Cloud Deployment using AWS, Azure, or Google Cloud for global accessibility and scalability.